

# Greedy Weighted Graph Spanners

## Implementation and Experimental Evaluation

Mini-Project on Embeddings of Graphs

Ben-Gurion University of the Negev

By: Alon Ganot, 209397512

Tomer Akdam, 318978236

Date: June 2026

Source paper: I. Althofer, G. Das, D. Dobkin, D. Joseph, and J. Soares, "On Sparse Spanners of Weighted Graphs", Discrete & Computational Geometry, 1993.

## Abstract

This report presents an implementation and empirical evaluation of the greedy weighted graph spanner algorithm of Althofer, Das, Dobkin, Joseph, and Soares [1]. The algorithm processes weighted edges in nondecreasing order and keeps an edge only when the current spanner does not yet contain a sufficiently short replacement path. The implementation is written in Java and uses Dijkstra shortest paths during construction, Kruskal MST computations for weight baselines, and deterministic graph generators for reproducible experiments.

The experimental evaluation separates structural graph-family experiments from larger density/scaling experiments. The structural benchmark contains 54 scenarios and 270 runs over  $k=1,2,3,4,5$ . The large benchmark contains 40 scenarios and 200 runs, including connected Erdos-Renyi-style weighted graphs with explicit probabilities  $p=0.02,0.05,0.10,0.20,0.30$ , large weighted grids, and complete weighted graphs. No stretch violations occurred in either benchmark. In the large benchmark, average edge reduction increased from 65.657% at  $k=1$  to 87.873% at  $k=5$ , while average lightness dropped from 9.645 to 1.034 times MST weight.

## Table of Contents

1. Project Requirements and Scope
  2. Mathematical Background and Paper Summary
  3. Implemented Algorithm
  4. Correctness Arguments
  5. Software Design
  6. Experimental Methodology
  7. Experimental Results
  8. Discussion
  9. Conclusions
  10. Future Work
- References
- Appendix A. Reproducibility Commands

# 1. Project Requirements and Scope

The mini-project requires reading a paper on low-distortion graph embeddings or spanners, implementing one algorithm from the paper, running experiments, and summarizing the results in a final report. The selected paper is Althofer et al., "On Sparse Spanners of Weighted Graphs" [1]. The implemented algorithm is the paper's greedy SPANNER construction for positive weighted undirected graphs.

The project studies sparse subgraphs called spanners. Given a connected weighted graph  $G = (V, E, w)$ , the goal is to construct a subgraph  $H = (V, E_H, w)$  on the same vertex set that approximately preserves all pairwise shortest-path distances. The term greedy weighted spanner construction is defined formally in Section 3, before the correctness proof ideas.

For a path  $P = (v_0, v_1, \dots, v_l)$ , its weighted length is the sum of the weights of its edges:

$w(P) = \sum_{i=0}^{l-1} w(v_i, v_{i+1})$ . The weighted distance  $d_G(u, v)$  is the minimum weighted length of any  $u, v$  path in  $G$ . All edge weights in this project are positive and finite.

- Algorithmic requirement: implement the weighted greedy spanner construction.
- Experimental requirement: evaluate the algorithm on several graph families, graph sizes, densities, and stretch parameters.
- Report requirement: explain the theory, implementation decisions, measured results, limitations, and possible future work.

# 2. Mathematical Background and Paper Summary

## 2.1 Spanner definition and stretch

Let  $G = (V, E, w)$  be an undirected graph with positive edge weights. A subgraph  $H = (V, E_H, w)$ , where  $E_H$  is a subset of  $E$ , is an  $r$ -spanner of  $G$  if for every pair  $u, v$  in  $V$ :

$$d_H(u, v) \leq r \cdot d_G(u, v)$$

The parameter  $r$  is the stretch factor. The  $r$ -stretch guarantee means that every distance in  $H$  is at most  $r$  times the corresponding distance in  $G$ . Since  $H$  is a subgraph of  $G$ , distances in  $H$  can only stay the same or become longer. Therefore the only possible distortion is an increase in shortest-path distance after edges are removed.

## 2.2 Sparsity and lightness

The paper distinguishes between two types of sparsity. Size sparsity concerns the number of edges in the spanner,  $|E_H|$ . Weight sparsity concerns the total edge weight  $w(H) = \sum_{e \in E_H} w(e)$ . The natural baseline for weight is the minimum spanning tree  $\text{MST}(G)$ , because an MST is the lightest connected spanning subgraph. Lightness is measured as  $\frac{w(H)}{w(\text{MST}(G))}$ .

An MST alone is often too sparse to preserve distances well. For example, removing one edge from an  $n$ -cycle creates a path in which the endpoints of the removed edge become distance  $n-1$  apart instead of distance 1. A spanner adds enough non-tree edges to control stretch while still keeping the graph much sparser than  $G$ .

### 2.3 Guarantees from Althofer et al.

Althofer et al. define  $SPANNER(G, r)$ , prove that its output is an  $r$ -spanner, and show that the output is sparse in both size and weight [1]. The paper states the main theorem using a parameter  $t$  and stretch  $r = 2t + 1$ . This project uses the implementation parameter  $k$  and sets  $r = 2k - 1$ . Therefore, when applying a theorem stated directly in the paper with  $r = 2t + 1$ , the corresponding value is  $t = k - 1$ .

The CSV output also reports a standard empirical normalization scale  $n^{1+\frac{1}{k}}$  for  $(2k - 1)$ -spanner experiments. This scale is useful for comparing runs, but the report does not quote it as the literal theorem statement without explaining the parameter mapping.

| Concept            | Project notation                   | Meaning in the report                                 |
|--------------------|------------------------------------|-------------------------------------------------------|
| Graph              | $G = (V, E, w)$                    | Input weighted undirected graph with positive weights |
| Spanner            | $H = (V, E_H, w)$                  | Subgraph on the same vertices with $E_H$ subset $E$   |
| Stretch            | $r = 2k - 1$                       | Allowed multiplicative distance distortion            |
| Distance guarantee | $d_H(u, v) \leq r \cdot d_G(u, v)$ | The $r$ -stretch guarantee                            |
| Lightness          | $\frac{w(H)}{w(MST(G))}$           | Weight relative to a minimum spanning tree            |

Table 1. Main notation used in the report.

## 3. Implemented Algorithm

The greedy algorithm starts with an empty graph  $H$  on the same vertex set as  $G$ . It processes all edges from lightest to heaviest. When considering an edge  $e = (u, v)$ , the algorithm computes the current shortest-path distance  $d_H(u, v)$  in the partial spanner. If the current path is already short enough, the edge is rejected. Otherwise, the edge is added to  $H$  with its original weight.

The current distance  $d_H(u, v)$  is computed with Dijkstra's algorithm on the current spanner  $H$ . If  $u$  and  $v$  are disconnected in  $H$ , the distance is treated as infinity, so the edge is accepted. This is why the output remains connected whenever the input is connected.

Pseudocode:

`GreedyWeightedSpanner( $G, k$ ):`

```

     $r = 2k - 1$ 
    sort all edges by nondecreasing weight
     $H$  = graph with  $V(G)$  and no edges
    for each edge  $e=(u, v)$  in sorted order:
        compute  $d_H(u, v)$  using Dijkstra on the current  $H$ 
        if  $d_H(u, v) > r \cdot w(e)$ :
            add  $e$  to  $H$  with its original weight
    return  $H$ 

```

The comparison is strict. If  $d_H(u, v)$  equals  $r \cdot w(e)$ , the existing path already satisfies the requested guarantee, so the edge is not needed.

### 3.1 Running time

Let  $n = |V|$  and  $m = |E|$ . Sorting the edges costs  $O(m \log m)$ . The implementation then runs Dijkstra once for each candidate edge. With a standard heap-based implementation, each call costs  $O((n + m_H) \log n)$ , where  $m_H$  is the current number of spanner edges and  $m_H \leq m$ . A clear worst-case bound for construction is therefore  $O(m(n + m) \log n)$ .

The reported runtime\_ms column measures only the greedy construction time. It excludes plotting, CSV post-processing, and report generation. Stretch and MST metrics are computed after construction and are not included in runtime\_ms.

### 3.2 Behavior at $k=1$

For  $k = 1$  the stretch bound is  $r = 1$ . This means exact preservation of all shortest-path distances, not necessarily returning the original graph. In weighted graphs, an edge can be removed if an equal or shorter path already exists.

In unit-weight complete graphs the behavior is different. Every direct edge has length 1, and every alternative path has length at least 2. Therefore  $k = 1$  keeps all complete-unit edges, while  $k \geq 2$  allows the output to collapse to a tree-like spanner in the tested cases.

## 4. Correctness Arguments

The paper's proof is based on three main ideas. They are presented after the algorithm because each argument refers to a decision made during the greedy construction.

1. Stretch: if an edge  $e = (u, v)$  is rejected,  $H$  already contains a replacement path from  $u$  to  $v$  of length at most  $r \cdot w(e)$ . Replacing every rejected edge on any shortest  $G$ -path by its stored replacement path gives  $d_H(a, b) \leq r \cdot d_G(a, b)$ .
2. Size sparsity: if the output contained a short cycle, the last edge of that cycle processed by weight would have had a short enough alternate path and would have been rejected. Thus the output has large girth, and extremal graph bounds imply sparsity.
3. MST containment: when an edge connects two disconnected components in the current  $H$ , the distance between its endpoints is infinity, so the edge is accepted. This mirrors Kruskal's algorithm and implies that the output contains at least one MST of the input graph. With equal-weight ties, the correct claim is containment of some MST, not necessarily the particular MST returned by an arbitrary library call.
4. Planar bound: in planar graphs, Euler-formula and face-size arguments give a sharper linear edge bound for the greedy output [1].

## 5. Software Design

The project is implemented in Java with JGraphT for weighted graph storage, Dijkstra shortest paths, and Kruskal MST computations. Python is used only for plotting and rendering report figures from Java-generated CSV files.

| Component                  | Role                                                                                                                                                              |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GreedyWeightedSpanner.java | Core greedy construction; validates $k$ and positive weights; computes Dijkstra on current $H$ .                                                                  |
| GraphMetrics.java          | Computes total weight, MST weight, maximum observed stretch, stretch violations, and tie-safe MST containment.                                                    |
| GraphFactory.java          | Creates deterministic graph families: paths, cycles, random trees, complete graphs, random connected graphs, grids, Erdos-Renyi-style graphs, and theta examples. |
| ExperimentRunner.java      | Runs quick, structural, and large experiment suites for $k=1..5$ and writes CSV rows.                                                                             |
| VisualExampleExporter.java | Exports Java-computed visual-example data so Python only renders already-computed graph/spanner memberships.                                                      |
| make_plots.py              | Creates derived metrics, summary tables, quantitative plots, density plots, runtime plots, and visual examples.                                                   |

*Table 2. Main project components.*

The sanity tests check that a heavy triangle edge is skipped, trees remain unchanged,  $k = 1$  preserves exact distances, an MST is contained in the output, stretch bounds hold on deterministic random graphs, and invalid weights are rejected. These tests are not the full experiment suite, but they are a fast correctness gate before running the report experiments.

## 6. Experimental Methodology

### 6.1 Structural benchmark

The structural benchmark uses  $k=1,2,3,4,5$  over 54 deterministic graph scenarios, giving 270 rows. It is designed to compare behavior across graph families rather than to serve as a clean runtime benchmark. The scenarios intentionally mix different graph sizes and densities.

| Family                              | Definition and purpose                                                                              |
|-------------------------------------|-----------------------------------------------------------------------------------------------------|
| Paths and trees                     | Sparse baselines. No edge can be removed without disconnecting the graph.                           |
| Cycles and cycle-threshold examples | Show when the final cycle edge becomes skippable under the allowed stretch.                         |
| Complete weighted graphs            | Dense weighted inputs where greedy sparsification should be strongest.                              |
| Complete unit graphs                | Equal-weight complete graphs; $k=1$ keeps all edges, $k \geq 2$ collapses to a tree-like spanner.   |
| Random sparse                       | Random tree plus roughly $n$ extra edges.                                                           |
| Random medium                       | Random tree plus roughly $3n$ extra edges, including repeated seeds for stable averages.            |
| Random dense                        | Random tree plus many extra edges, still below complete density to keep runtime moderate.           |
| Grid planar                         | Structured planar weighted graphs connected to the paper's planar theorem.                          |
| Theta hard                          | Many internally-disjoint unit chains between two terminals; hard near its target stretch.           |
| Wide weights                        | Random/complete graphs with weights sampled from a wider range to study sensitivity to heavy edges. |

Table 3. Structural graph families and their role.

### 6.2 Large density/scaling benchmark

The large benchmark answers the instructor's request for larger graphs and controlled densities. It contains 40 scenarios and 200 runs. The main random family is a connected Erdos-Renyi-style graph: first a random spanning tree is added to guarantee connectivity, then every remaining possible edge is added independently with probability  $p$ , and every edge receives a positive integer weight.

| Large family                     | Parameters                                                                                                                             |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Connected weighted random graphs | $p \in \{0.02, 0.05, 0.10, 0.20, 0.30\}$ ; sizes up to $n = 800$ for lower probabilities and up to $n = 400$ for higher probabilities. |
| Large weighted grids             | 20x20 and 30x30 planar grids.                                                                                                          |
| Large complete weighted graphs   | $n = 75$ and $n = 100$ complete graphs.                                                                                                |

Table 4. Large density/scaling benchmark families.

### 6.3 Metrics

- Edge reduction:  $100 \cdot \left(1 - \frac{|E_H|}{|E_G|}\right)$ . Higher values mean more original edges were removed.
- Lightness:  $\frac{w(H)}{w(MST(G))}$ . Values close to 1 mean the spanner is close to MST weight.
- Maximum observed stretch: maximum over all vertex pairs of  $\frac{d_H(u,v)}{d_G(u,v)}$ .
- Stretch violations: number of vertex pairs whose observed stretch exceeds  $2k - 1$ .
- General size-bound ratio:  $|E_H|$  divided by an empirical  $n^{(1+\frac{1}{k})}$  normalization scale.
- Planar size-bound ratio: for known planar families and  $k \geq 2$ ,  $|E_H|$  divided by  $(n - 1) \cdot \left(1 + \frac{1}{k-1}\right)$ .
- Runtime: construction time of the greedy spanner only, measured around `GreedyWeightedSpanner.buildSpanner`.

A replacement path is an existing path in the current spanner  $H$  between the endpoints of a candidate edge. If its length is at most the allowed threshold, the direct edge can be rejected because  $H$  already has a sufficient substitute.

## 7. Experimental Results

### 7.1 Structural benchmark: summary by $k$

The structural benchmark had no stretch violations in any of the 270 runs. Larger  $k$  gives the algorithm more room to use replacement paths, so the average number of selected edges and the average lightness both decrease. The edge reduction average stops around 50% because the benchmark intentionally includes paths, trees, cycles, and theta-style graphs that are already sparse or deliberately hard to sparsify.

| $k$ | $2k - 1$ | avg $ E_H $ | avg edge reduction % | avg $\frac{w(H)}{w(MST)}$ | maximum observed stretch | avg runtime ms |
|-----|----------|-------------|----------------------|---------------------------|--------------------------|----------------|
| 1   | 1        | 147.54      | 24.24                | 3.909                     | 1.000                    | 2.204          |
| 2   | 3        | 78.61       | 43.50                | 1.224                     | 3.000                    | 1.352          |
| 3   | 5        | 71.93       | 46.40                | 1.065                     | 5.000                    | 1.000          |
| 4   | 7        | 70.04       | 47.89                | 1.024                     | 7.000                    | 0.944          |
| 5   | 9        | 69.20       | 48.56                | 1.009                     | 9.000                    | 0.907          |

Table 5. Structural benchmark summary over 54 scenarios and 270 total rows.

#### Structural benchmark: sparsity and weight tradeoff

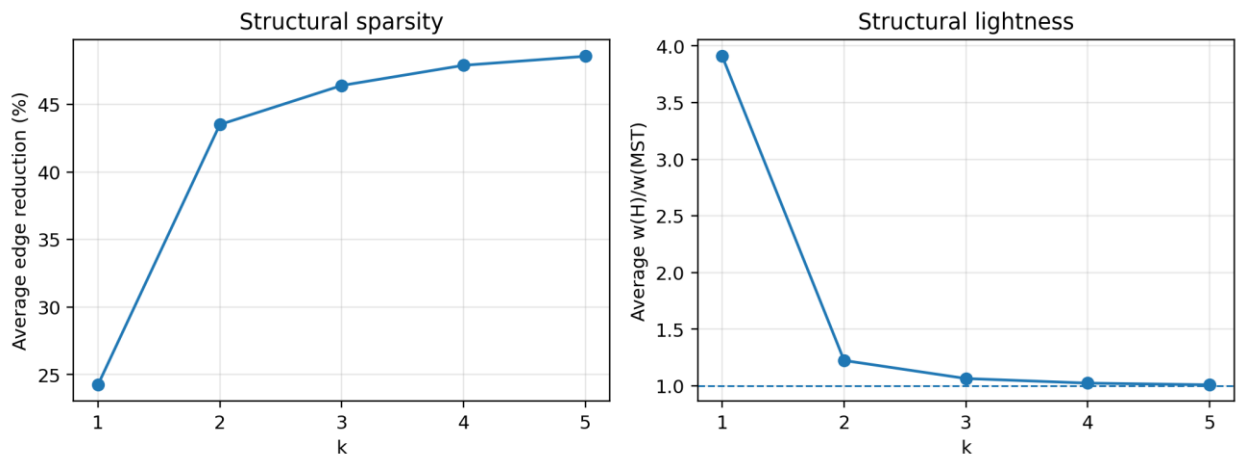


Figure 2. Structural benchmark: average edge reduction and lightness as  $k$  increases.



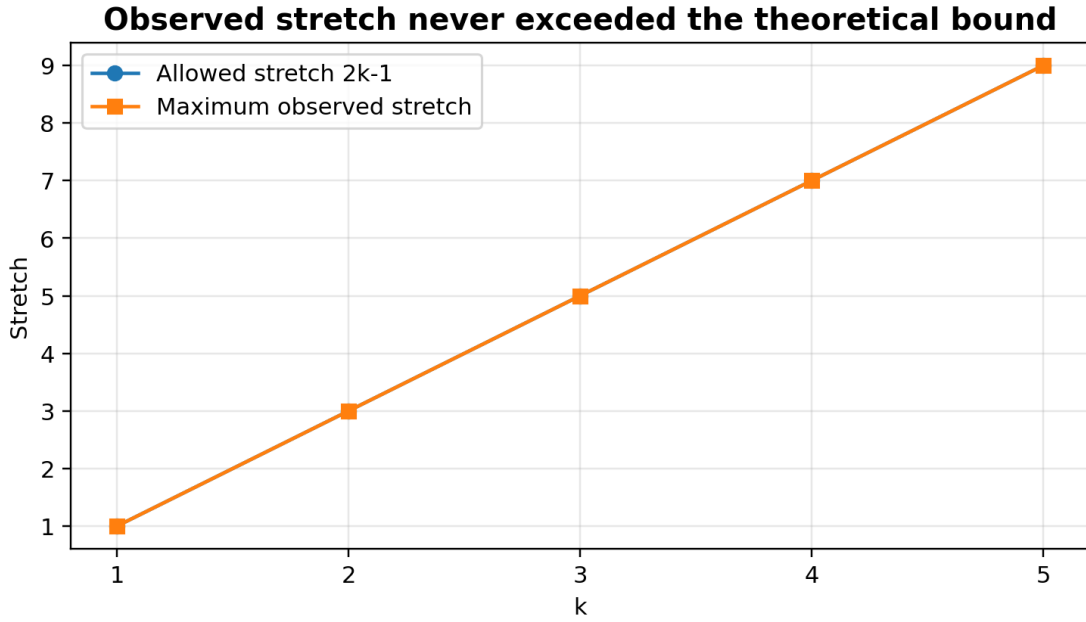


Figure 3. Maximum observed stretch compared with the theoretical bound  $2k-1$ .

## 7.2 Structural behavior by graph family

The family results match the expected theory and intuition. Dense graphs have many alternate paths and therefore lose many edges. Trees and paths do not shrink because any removed edge disconnects the graph. Theta-style lower-bound examples resist sparsification around their target stretch because every chain edge is difficult to replace by a short alternative route.

Complete unit graphs require special interpretation. At  $k = 1$  all unit edges are required for exact pairwise distances, so no edge is removed. At  $k \geq 2$ , an alternative two-edge path is short enough for many direct unit edges, and the output collapses to  $n - 1$  edges in the tested complete-unit cases.

| Family            | Scenarios | avg $ E_G $ | avg $ E_H $ | avg edge reduction % | avg $\frac{w(H)}{w(MST)}$ | max stretch |
|-------------------|-----------|-------------|-------------|----------------------|---------------------------|-------------|
| Complete weighted | 6         | 579.3       | 31.50       | 92.05                | 1.016                     | 5.000       |
| Complete unit     | 2         | 485.0       | 29.00       | 92.50                | 1.000                     | 2.000       |
| Cycle             | 4         | 85.0        | 85.00       | 0.00                 | 1.017                     | 1.000       |
| Grid planar       | 3         | 185.3       | 110.0       | 39.01                | 1.145                     | 5.000       |
| Path              | 4         | 84.0        | 84.00       | 0.00                 | 1.000                     | 1.000       |
| Random dense      | 4         | 489.0       | 71.25       | 85.52                | 1.045                     | 4.900       |
| Random medium     | 11        | 366.3       | 95.45       | 74.00                | 1.081                     | 5.000       |
| Random sparse     | 3         | 205.7       | 110.3       | 46.93                | 1.134                     | 5.000       |
| Theta hard        | 4         | 90.0        | 85.25       | 7.92                 | 1.184                     | 5.000       |
| Tree              | 4         | 84.0        | 84.00       | 0.00                 | 1.000                     | 1.000       |

Table 6. Structural family highlights at  $k=3$ .

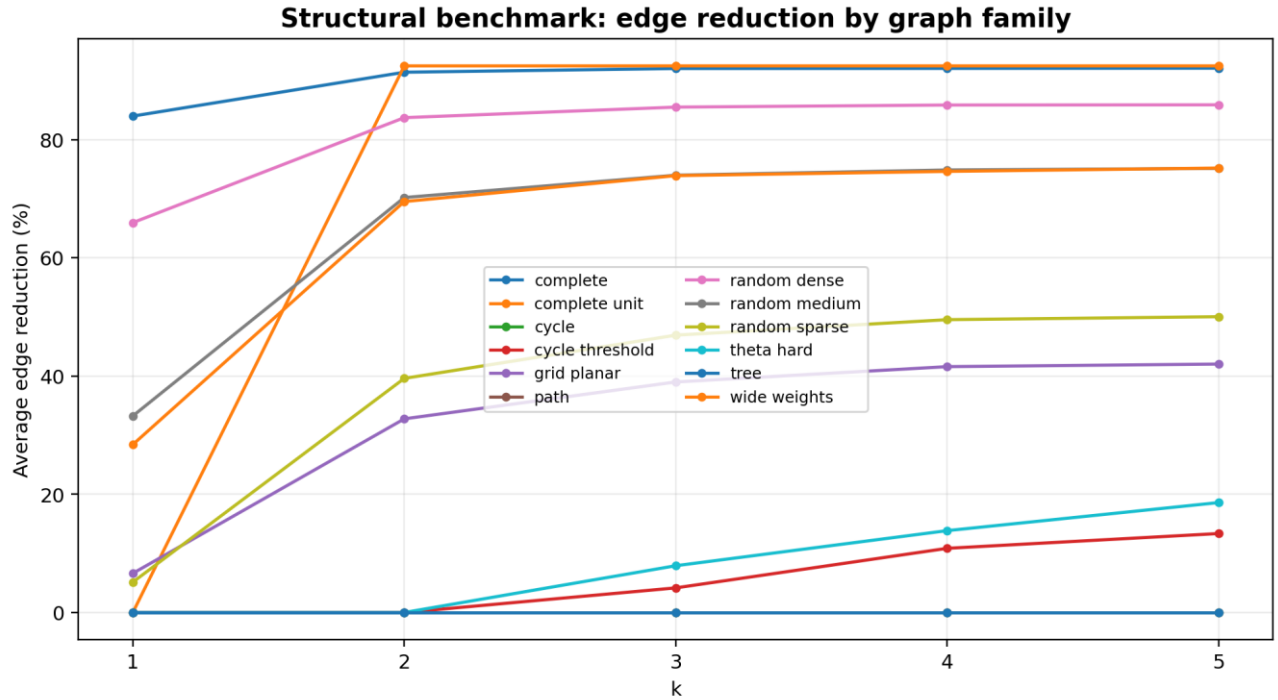


Figure 4. Edge reduction by selected graph family. The result is expected: dense families shrink, minimally connected families do not.

### 7.3 Planar benchmark

A planar graph is a graph that can be drawn in the plane without edge crossings. The paper proves a sharper size guarantee for planar weighted graphs. In the report experiments, the planar ratio is computed only for generated families known to be planar: paths, cycles, trees, grids, and theta-style examples.

For  $k \geq 2$  the project computes  $|E_H| / (n - 1) \cdot \left(1 + \frac{1}{k-1}\right)$ . Ratio below 1 means the measured spanner size is below the planar theorem bound after applying the parameter mapping. The tested planar families stayed below ratio 1 in all reported averages.

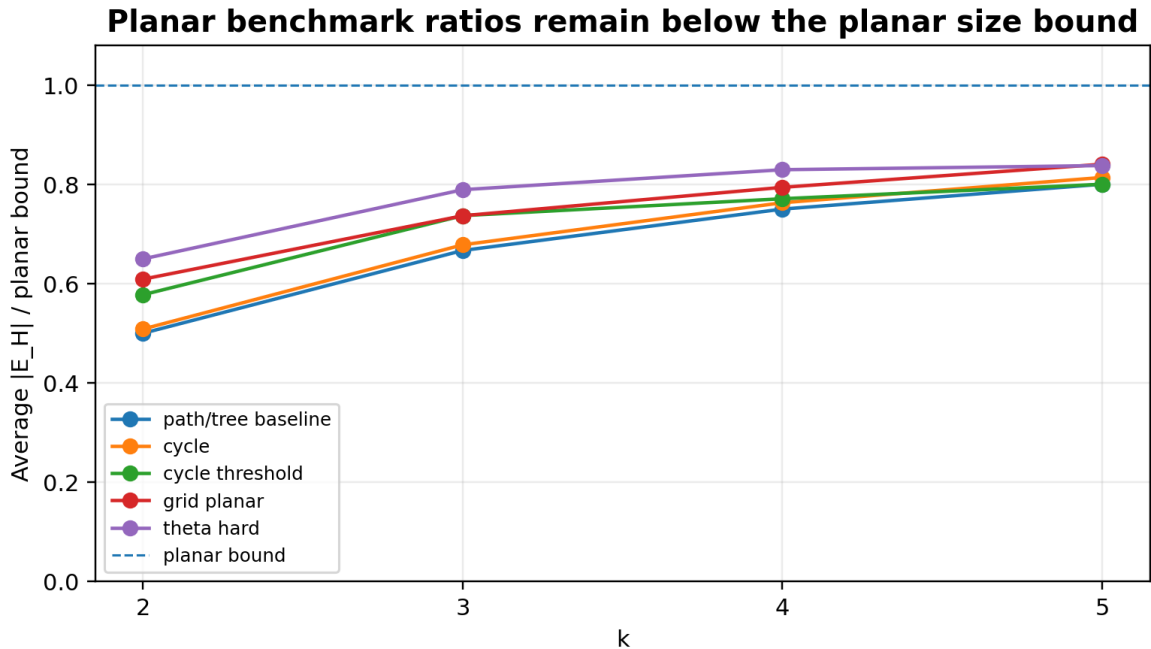


Figure 5. Ratio to the planar size bound for known planar benchmark families.

## 7.4 Visual examples

The visual examples compare the original graph  $G$  and the greedy spanner  $H_k$  on the same vertex layout. Blue edges are a chosen MST contained in  $H_k$ , red edges are additional spanner edges, and pale gray edges are removed original edges. The Java exporter computes all edge memberships; Python only renders the already-computed data.

| Example      | k | n  | $ E_G $ | $ E_H $ | reduction % | max stretch | bound | $\frac{w(H)}{MST}$ |
|--------------|---|----|---------|---------|-------------|-------------|-------|--------------------|
| Complete n10 | 2 | 10 | 45      | 10      | 77.78       | 2.091       | 3.000 | 1.214              |
| Random n14   | 3 | 14 | 35      | 13      | 62.86       | 3.000       | 5.000 | 1.000              |
| Grid 5x5     | 3 | 25 | 40      | 24      | 40.00       | 4.500       | 5.000 | 1.000              |
| Theta K3     | 3 | 20 | 24      | 24      | 0.00        | 1.000       | 5.000 | 1.263              |
| Cycle n7     | 4 | 7  | 7       | 6       | 14.29       | 6.000       | 7.000 | 1.000              |
| Unit K20     | 2 | 20 | 190     | 19      | 90.00       | 2.000       | 3.000 | 1.000              |

Table 7. Metrics for the generated visual examples.

### Representative visual graph examples

Left: original input graph. Right: greedy spanner on the same layout. Blue = chosen MST edges, red = additional spanner edges, pale gray = removed edges.

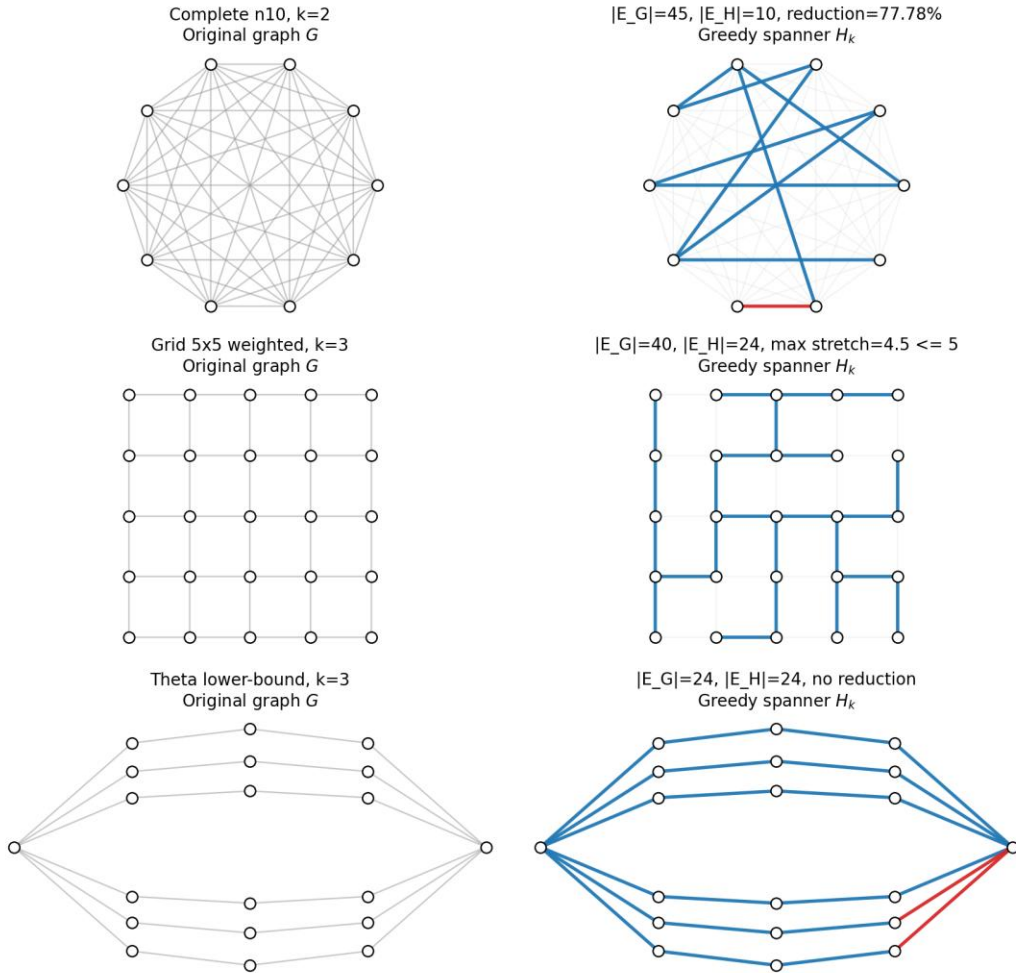


Figure 6. Representative visual examples. Each row compares an original input graph  $G$  with the greedy spanner  $H$  on the same vertex layout.

The complete-graph example shows strong sparsification in a dense weighted input: only a small connected weighted backbone is kept while most direct edges become redundant.

The grid example shows structured planar behavior. The spanner keeps a connected set of short replacement paths and removes heavier grid edges where the stretch bound remains valid.

The theta example shows the opposite behavior. Its many internally-disjoint chains make edge removal difficult at the target stretch, so the greedy construction keeps all original edges.

## 7.5 Large density/scaling benchmark

The large benchmark directly addresses performance and density behavior on larger graphs. It has 200 rows over 40 scenarios and again produced zero stretch violations. It also gives a better basis for discussing runtime than the structural benchmark, because it focuses on larger inputs and controlled density parameters.

| k | $2k - 1$ | avg $ E_H $ | avg edge reduction % | avg $\frac{w(H)}{w(MST)}$ | maximum observed stretch | avg runtime ms |
|---|----------|-------------|----------------------|---------------------------|--------------------------|----------------|
| 1 | 1        | 1339.03     | 65.657               | 9.645                     | 1.000                    | 651.925        |
| 2 | 3        | 539.10      | 84.414               | 1.710                     | 3.000                    | 419.850        |
| 3 | 5        | 434.53      | 86.889               | 1.194                     | 5.000                    | 366.700        |
| 4 | 7        | 404.38      | 87.571               | 1.077                     | 7.000                    | 345.300        |
| 5 | 9        | 392.20      | 87.873               | 1.034                     | 9.000                    | 333.975        |

Table 8. Large benchmark summary over 40 scenarios and 200 total rows.

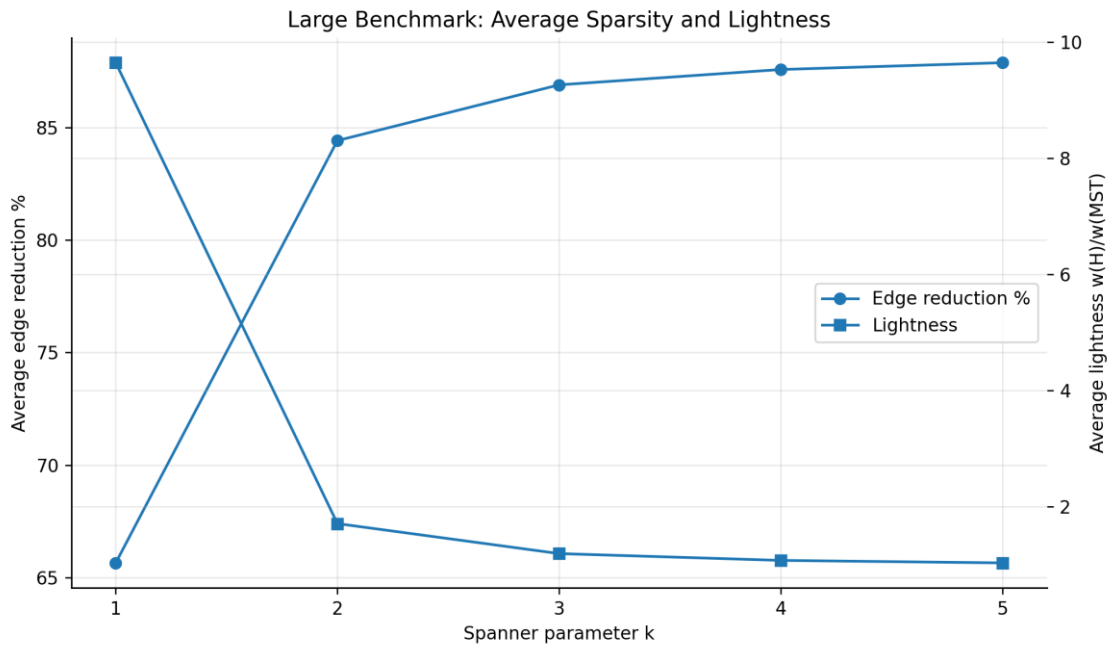


Figure 7. Large benchmark: average edge reduction increases and average lightness decreases as  $k$  grows.

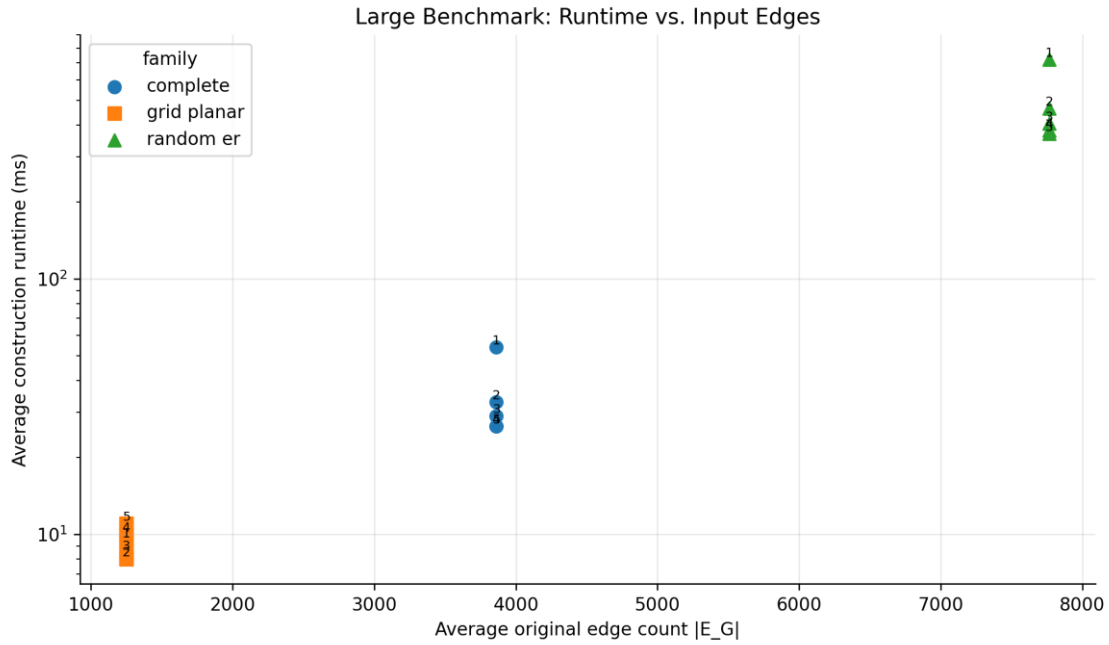


Figure 8. Large benchmark: construction runtime versus average original edge count. Labels mark the  $k$  value for each family point.

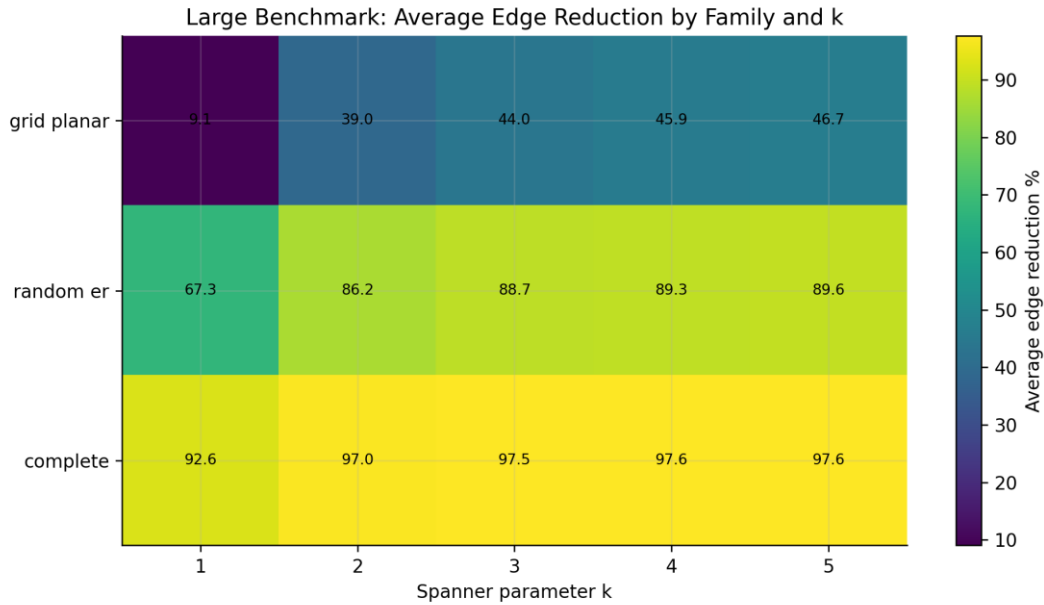


Figure 9. Large benchmark: average edge reduction by graph family and  $k$ .

The large benchmark plots focus on density and scaling. Figure 7 summarizes the  $k$ -dependent sparsity and lightness trend, Figure 8 reports construction time against input edge count, and Figure 9 separates the main large benchmark families.

The large complete graphs illustrate the extreme dense case: complete\_n100 has 4,950 input edges and at  $k = 3$  the spanner keeps 108 edges, a 97.82% reduction, with maximum observed stretch exactly 5 and no violations. The 30x30 weighted grid has 900 vertices and 1,740 input edges; at  $k = 3$  it keeps 972 edges, a 44.14% reduction, with maximum observed stretch 4.989 under the allowed bound 5.

## 8. Discussion

### 8.1 What the experiments confirm

- Correctness: all measured pairs satisfied  $d_H(u, v) \leq (2k - 1)d_G(u, v)$  in both structural and large benchmarks.
- Sparsity/stretch tradeoff: increasing  $k$  allows longer replacement paths and therefore removes more edges on average.
- Weight behavior: for  $k \geq 3$ , both structural and large benchmarks produced average lightness close to 1, especially compared with  $k = 1$ .
- Graph family matters: dense graphs shrink dramatically, while paths, trees, and lower-bound-style examples do not.
- The structural benchmark is best for explaining graph-family behavior; the large benchmark is best for density and runtime discussion.

### 8.2 Runtime interpretation

The implementation recomputes Dijkstra for every candidate edge. This is simple and faithful to the greedy definition, but it is not optimized for large-scale performance. The large benchmark demonstrates that the implementation is still practical for the project sizes: even across denser random graphs and complete graphs up to  $n = 100$ , average construction times were below one second for  $k \geq 2$  and about 652 ms for  $k = 1$ . Runtime decreases with  $k$  in the large average because larger  $k$  tends to accept fewer edges earlier, so the current spanner  $H$  is smaller during later Dijkstra calls. This is empirical behavior of the tested implementation and should not be stated as a general theorem.

### 8.3 Limitations

- The construction prioritizes clarity over asymptotic speed. Faster implementations would require more sophisticated dynamic shortest-path handling or alternative spanner algorithms.
- The large benchmark is larger than the structural one, but it is still a mini-project benchmark rather than a production-scale performance study.
- The planar experiments use generated families known to be planar. The code does not perform a general planarity test for arbitrary inputs.
- The reported lightness is empirical. It should not be overclaimed as proof of a stronger theorem than the paper provides.

## 9. Conclusions

The project satisfies the mini-project requirements: it studies a spanner paper, implements a central algorithm from that paper, performs structural and large-scale experiments, and summarizes the results. The greedy weighted spanner algorithm is simple but effective. It preserves the required stretch bound in all tested cases and often produces spanners that are both sparse and close to MST weight.

The strongest empirical pattern is that dense and weighted graphs benefit heavily from the greedy filter, while minimally connected or lower-bound-style graphs do not. This matches the theory: a spanner can only remove edges that have sufficiently short replacement paths.

## 10. Future Work

- Optimize the construction by avoiding a full Dijkstra recomputation for every candidate edge, or compare against faster known spanner algorithms such as later linear-time constructions.
- Add still larger graphs and more repetitions for statistically stronger runtime curves.
- Add automatic planarity testing so arbitrary generated or imported graphs can be classified before applying planar-bound comparisons.
- Compare greedy weighted spanners with randomized or clustering-based spanner algorithms from the broader graph-spanner literature, including Peleg and Schaffer [2].
- Study how different weight distributions affect lightness and observed stretch, not only uniform integer weights.

## References

- [1] I. Althofer, G. Das, D. Dobkin, D. Joseph, and J. Soares. "On Sparse Spanners of Weighted Graphs." *Discrete & Computational Geometry*, 9:81-100, 1993.
- [2] D. Peleg and A. Schaffer. "Graph Spanners." *Journal of Graph Theory*, 13(1):99-116, 1989.

## Appendix A. Reproducibility Commands

From the project root on Windows PowerShell, reproduce the full report results with:

```
.\scripts\test.ps1; .\scripts\run_experiments.ps1 structural; .\scripts\make_plots.ps1 structural;  
.\scripts\run_experiments.ps1 large; .\scripts\make_plots.ps1 large
```

Structural outputs are written under results/plots\_structural. Large density/scaling outputs are written under results/plots\_large.